

Research Document

IoT Measurement Setup

C. Stijlaart

Table of Contents

1. Database	1
1.1. Local- or Cloud-based Database	1
1.2. Database Structure	2
2. Data Structure	3
2.1. Data Structure Techniques	4
2.2. Recommendation	5

1. Database

A database can be stored in multiple manners. Each manner having their own advantages and disadvantages.

1.1. Local- or Cloud-based Database

A database can be stored in two manners, locally or in the cloud. Both of these options having their own advantages and disadvantages, making an option more suitable for one purpose than another.

1.1.1. Local Database

Local databases refer to databases that are hosted and managed on-premises, typically within an organization's physical infrastructure. They are stored and maintained locally on the organization's servers or dedicated hardware.

Advantages

1. **Data Control:** The organization has full control over their data. Custom security measures and access controls can be set to their specific needs.
2. **Performance:** As the data is stored locally, the data can be accessed faster with a lower latency.
3. **Compliance:** Since data is stored locally, the data is secured by the better compliance options as the data does not leave the premises of the organization.

Disadvantages

1. **Cost:** As everything is locally, there is a dependency on hardware and staff for management. This can be costly for smaller projects.
2. **Scalability:** When large sizes of data is stored, the storage will run low and therefor must be expanded upon. This requires a significant investment and updates in software to manage the drives.
3. **Security:** While local databases offer control, they also demand more responsibility for security. There must be invested in a robust security to protect against physical and cyber threats.

1.1.2. Cloud-based Database

Cloud-based databases are hosted and managed by third-party cloud service providers, such as Amazon Web Services (AWS), Microsoft Azure, or Google Cloud Platform. Data is stored and processed on remote servers maintained by these providers.

Advantages

1. **Scalability:** Cloud databases allow for easier scalability. The amount of storage and computing can be changed on-demand, ensuring there will always be enough storage.
2. **Accessibility:** As they are stored in the cloud, data can be accessed from anywhere with the help of an internet connection. This allows for more flexibility and possible remote work and

global collaborations.

3. **Automatic Updates:** Cloud providers handle updates, patches, and maintenance, reducing the burden on in-house IT teams. This ensures databases are running with the latest security features.

Disadvantages

1. **Data Security:** While cloud providers ensures the good security, a third party is involved. Meaning that the data is entrusted to the third party. Therefore, it's crucial to carefully manage access control, encryption, and compliance to mitigate potential risks.
2. **Data Transfer Costs:** Transmitting large volumes of data to and from the cloud can incur additional costs, especially if the organization has limited bandwidth or significant data transfer needs.
3. **Vendor Lock-In:** Moving away from a cloud provider can be complex and costly, creating a form of vendor lock-in. It's essential to plan for data portability and exit strategies.

1.1.3. Recommendation Database Storage

As the goal of the project is to create a prototype, there are certain aspects that are not yet relevant regarding to the database. A good example of such an aspect is the security and scalability. For a prototype a local database is a solid start. It allows for full control of the data, no internet dependency and is easier to setup.

1.2. Database Structure

As for the database itself, it has one main requirement, flags. Data must be retrievable based upon, so called, flags. These flags allow for data sets to be accessible for only the groups that have access to given flag. Another important aspect to be taken into account is the uncertainty of what data fields will be stored within When it comes to the type of data structure, there are two options, relational and non-relational.

1.2.1. Relational Databases (RDBMS)

A relational database is structured, meaning the data is organized in tables. Many times, the data within these tables have relationships with one another or dependencies.

Advantages

1. **Structured Data:** RDBMS excels in handling structured data with well-defined schemas.
2. **Data Integrity:** ACID properties ensure strong data integrity and consistency, making RDBMS suitable for applications where data accuracy is critical.
3. **Historical Data:** RDBMS systems are well-suited for creating a historical record of changes of changes made to data over time.

Disadvantages

1. **Rigid Schema:** The fixed schema of RDBMS can be limiting if your data sets evolve over time or if you need to store unstructured or semi-structured data.
2. **Scalability:** Scaling RDBMS systems can be challenging, especially for high-velocity data streams, which may be a limitation for real-time data collection.

1.2.2. Non-Relational Databases (NoSQL)

A non-relational database is document-oriented, meaning all information gets stored in more of a laundry list order. You will have all of your data listed within a single construct or document.

Advantages

1. **Flexible Schema:** NoSQL databases allow for a dynamic or flexible schema, making them suitable for storing data sets with varying or evolving data fields.
2. **Scalability:** NoSQL databases are designed for horizontal scalability, making them well-suited for real-time applications with rapidly changing data.
3. **High Throughput:** NoSQL databases can handle high volumes of data and can be optimized for high throughput, crucial for real-time data capture.

Disadvantages

1. **Eventual Consistency:** The BASE model, followed by NoSQL databases, may provide eventual consistency, which may not be suitable if strong data consistency is essential.
2. **Learning Curve:** Depending on the chosen NoSQL database type (e.g., document-oriented, key-value, graph), there may be a learning curve for adapting to the specific data model.

1.2.3. Recommendations Database Structure

Keeping further development for the database and data structure in mind, the recommendation is to make use of a non-relational as this allows for more flexibility. To clarify this, as the data sets may have fields added or removed dynamically, and considering the need for flexibility, scalability, a NoSQL database is more usable. An example of such is MongoDB, as its easy to adapt changes in data structure, capture real-time data, and scale your database as needed. MongoDB is a good recommendation for a NoSQL database that allows for encrypted data on the network and on disk

2. Data Structure

There are many types of data structures that can be used for a database. Each type is used for a different purpose. Examples of such are tables, indexes, trees and heaps. When picking a data structure it is of great importance to keep in mind in what type of database it is stored.

As mentioned in [Section 1.2.3](#), the recommendation is to make use of an local relational database. This local database will be stored on a drive. Based upon this, it is a given that there is made use of tables. While tables seem rather straight forward, different techniques can be used.

Techniques of such are normalization and indexes. These two techniques can enhance performance and overall clarity of the database.

2.1. Data Structure Techniques

2.1.1. Tables

In a relational database, a table is a structured data storage unit. It is a two-dimensional structure consisting of rows and columns. Each row represents a single record or data entity, and each column represents an attribute or field of that entity.

Purpose

Tables are used to organize and store data in a structured manner. They provide a way to represent different types of data entities and their attributes.

2.1.2. Normalization

Normalization is a database design technique used to minimize data redundancy and improve data integrity. It involves breaking down large tables into smaller, related tables and establishing relationships between them.

Purpose

Using this technique reduces data anomalies such as insertion, update, and deletion anomalies. It also ensures that data is organized in such a way that it minimizes redundancy, making it easier to maintain and update the database.

Levels of Normalization

There are several normal forms (e.g., First Normal Form, Second Normal Form, Third Normal Form, Boyce-Codd Normal Form) that define progressively stricter rules for organizing data. Each level addresses specific types of data redundancy and relationships between tables.

Examples

SensorId	Value	Type	Flag
1	0.11	float	chemist
2	2	int	business

SensorId	Sensor Device Id	Sensor Type
1	189302847	Temperature
2	73820173	Color

2.1.3. Indexes

An index in a relational database is a data structure that improves the speed of data retrieval

operations on a table. It's like an additional table containing a subset of the data in the main table, organized for quick lookup based on one or more columns.

Purpose

Indexes are used to speed up the retrieval of rows from a table. Without an index, the database would need to perform a full table scan to find specific rows, which can be slow for large datasets. Indexes allow the database engine to locate the desired rows more efficiently.

Types of Indexes There are various types of indexes, including B-tree indexes (commonly used for most data types), hash indexes (for exact matches), and bitmap indexes (useful for columns with a limited set of possible values).

2.2. Recommendation

For the prototype, the recommendation is to make use of a tables with the normalization technique. As a lot of different data comes in, each corresponding to a different view point, it can enhance the clarity of the data to be structured in such a manner. If feasible, indexes is a great technique to implement for better performance. Using indexes allow for data to be processed faster, meaning data can be received and retrieved in a continuos stream.